

Practical Primer for Designing a Photo Editor From Scratch

1. The central rule: a slider is a perceptual promise, not a raw variable

A user does not move “Brightness” expecting every RGB value to move linearly toward white. They expect the photo to feel brighter while still looking like a photograph. They expect skies not to blow out immediately, faces not to turn chalky, shadows not to become gray mush, and colors not to mutate.

So every slider needs three layers:

1. **The user contract:** what the slider appears to promise.
2. **The image operator:** what mathematical transform is applied.
3. **The guardrails:** how the transform is limited to avoid clipping, halos, hue shifts, skin damage, noise amplification, and ugly saturation.

A naive slider directly maps UI position to one parameter. A good slider maps UI position to a small coordinated edit recipe, often using image statistics.

For example:

- A bad Brightness slider: `rgb += value`.
- A decent Brightness slider: apply a gamma-like midtone curve.
- A good Brightness slider: lift midtones, preserve black and white endpoints, compress highlights, keep color ratios stable, attenuate the effect near already-clipped areas, and use robust image statistics so a dark concert photo and a snowy landscape respond differently.

Your editor should be built around this principle: **simple controls, smart operators, conservative failure modes.**

2. Internal pipeline: do not edit like an 8-bit paint program

Before designing individual sliders, set the editing pipeline correctly. Many slider problems come from clipping too early or working in the wrong space.

Use this as a baseline:

1. Decode the image into a floating-point working buffer.
2. Convert into a known working color space.
3. Keep values unclamped during the edit stack.
4. Do tone operations on luminance or log luminance, not directly and independently on R, G, and B.
5. Preserve hue by changing luminance and rescaling RGB ratios when possible.

6. Gamut-map or soft-clip only near the end.
7. Export to the target display space.

For a consumer editor, 32-bit float per channel is a good default. Even if the source is an 8-bit JPEG, the working buffer should not be 8-bit. Intermediate values should be allowed to go below 0 or above 1, because those values may later be recovered by tone compression, highlight rolloff, or output mapping.

Working spaces

Use two conceptual spaces:

Scene or linear RGB for exposure-like operations. Linear RGB means pixel values are proportional to light. Multiplying linear RGB by 2.0 is equivalent to increasing exposure by one stop.

Perceptual luminance/chroma space for color and appearance controls. OKLab/OKLCH is a practical modern choice. CIELab is also usable. HSV/HSL are convenient for UI widgets but are poor foundations for serious photo editing because equal numeric changes do not correspond well to equal perceived changes.

A practical compromise:

- Store the image in linear RGB.
- Compute luminance `Y` from linear RGB.
- For tone operations, operate on `log2(Y + epsilon)` or on a normalized perceptual tone value `x` in `[0, 1]`.
- For color operations, convert to OKLab/OKLCH, adjust chroma/hue/lightness, then convert back.

Never clip inside the stack

This rule matters more than almost anything else.

Do not do this after each slider:

```
rgb = clamp(rgb, 0, 1)
```

That destroys information and makes later sliders behave badly. Instead:

```
rgb_working = transform(rgb_working)
preview_rgb = output_transform_with_soft_clip(rgb_working)
```

Only the preview/export transform should clamp, and even then it should use a shoulder/toe curve or gamut mapping rather than blunt channel clipping.

3. Slider mapping: make the center precise and the extremes useful

A slider from to should not usually map linearly to the underlying parameter.

Use a perceptual mapping:

```
u = slider / 100
amount = sign(u) * abs(u) ^ 1.35
```

This gives more precision around zero and still reaches meaningful extremes. For some sliders, use or instead of if small changes are too jumpy.

Do not make a broken novelty setting. In a consumer editor, the far end of a slider should be strong but still plausible on at least some images. Hide destructive or laboratory-grade ranges behind an advanced mode.

Recommended default UI behavior:

- Normal drag: fine enough for visible but controlled changes.
- Shift-drag or keyboard arrows: smaller steps.
- Double-click/tap value: reset.
- Press-and-hold or keyboard shortcut: before/after.
- Show a subtle clipping warning only when useful, not constantly.

But the deeper rule is: **the slider should feel alive by ± 10 and still usable at ± 100** . If users move a slider halfway before anything happens, the mapping or operator is wrong.

4. Required image statistics

Before applying tone and color sliders, compute robust statistics. Do not base behavior on min/max pixel values; one specular highlight or dead pixel will wreck the edit.

Compute at least:

```
Y linear luminance
L = log2(Y + epsilon)

p0.1, p0.5, p1, p5, p25, p50, p75, p95, p99, p99.5, p99.9 of luminance
per-channel clipping ratios
saturation/chroma percentiles
skin likelihood mask, if available
sky likelihood mask, if available
```

```
noise estimate in dark regions
local contrast estimate
```

Use these to adapt sliders.

Examples:

- Positive Exposure should be attenuated if too many pixels are already near white.
- Positive Shadows should be attenuated in very noisy dark regions.
- Positive Saturation should do less to already saturated colors.
- Positive Clarity should do less on faces and smooth skies.
- Dehaze should do little when the image has no haze-like low-contrast bright veil.

This is what makes sliders feel “smart.”

5. Tone normalization: the hidden foundation

Most tone sliders should operate on a normalized tone coordinate:

```
x = normalized perceptual tone in [0, 1]
```

A simple version:

```
black_ref = percentile(logY, 0.5)
white_ref = percentile(logY, 99.5)
x = (logY - black_ref) / (white_ref - black_ref)
x = clamp_for_masking_only(x, 0, 1)
```

Do not use this clamped `x` as the only image value; use it to build masks and tone weights. The actual pixel math should still happen in float.

This normalized tone coordinate lets you define sliders by tonal region:

```
shadows:    x ≈ 0.00–0.35
darks:      x ≈ 0.15–0.50
midtone:    x ≈ 0.30–0.70
lights:     x ≈ 0.50–0.85
highlights: x ≈ 0.65–1.00
```

Use smooth masks, not hard thresholds:

```
w_shadows    = 1 - smoothstep(0.18, 0.45, x)
w_highlights  = smoothstep(0.55, 0.88, x)
w_midtones    = exp(-((x - 0.5)^2) / (2 * 0.22^2))
```

For local tone tools, build these masks from a blurred or edge-aware luminance image, not raw pixel luminance. Otherwise local texture gets flattened or exaggerated in ugly ways.

6. Exposure slider

User contract

Exposure should feel like changing camera exposure. It mainly changes overall light level, especially midtones, in stop-like units.

Recommended UI range

Consumer:

-2.0 EV to +2.0 EV

Advanced/raw mode:

-5.0 EV to +5.0 EV

Mapping

```
u = slider / 100
EV = 2.0 * sign(u) * abs(u)^1.35
gain = 2 ^ EV
linear_rgb *= gain
```

For JPEG/HEIC input, consider limiting the positive range slightly because there is less recoverable headroom than raw.

Guardrails

Positive exposure should not blindly blow highlights. Add a preview/output shoulder and optionally reduce the applied exposure when highlight clipping becomes excessive.

A simple protection rule:

```
if p99.5_after_output > 0.995 and slider > 0:  
    reduce EV until non-specular clipping is below target
```

Suggested clipping targets:

```
ordinary image:    < 0.1% non-specular pixels clipped  
high-key image:    < 0.3%  
night/neon image:  allow specular clipping, protect colored signs
```

Classify specular highlights loosely as tiny, very bright, low-detail regions. These may clip without the image feeling broken. Large areas of clipped sky, skin, clothing, or clouds should be protected.

Common failure modes

- Channel clipping causes hue shifts in highlights.
- Positive exposure turns skin chalky.
- Negative exposure makes the image muddy because black point is not adjusted.
- Exposure is doing the job Brightness should do.

7. Brightness slider

User contract

Brightness should make the photo read brighter or darker without feeling like exposure compensation. It should affect midtones strongly, shadows somewhat, highlights gently, and endpoints minimally.

This is the slider where naive implementations most often fail.

Do not implement as

```
rgb += k
```

or

```
rgb = mix(rgb, white, k)
```

Both are wrong for photos.

Recommended behavior

Operate on normalized tone .

For positive brightness:

$$x' = x + b * \sqrt{x} * (1 - x)$$

For negative brightness:

$$x' = x - b * x * \sqrt{1 - x}$$

Where:

$$b = 0.45 * \text{abs}(\text{slider} / 100)^{1.35}$$

Use the positive formula when the slider is above zero and the negative formula when below zero.

Why this works:

- Pure black stays near black.
- Pure white stays near white.
- Midtones move visibly.
- Highlights compress instead of clipping.
- Shadows open without becoming uniformly gray.

Better version

Use tone zones:

```
brightness_positive =  
    + midtone_lift  
    + gentle_shadow_lift  
    + highlight_compression  
  
brightness_negative =  
    - midtone_lowering  
    - gentle_shadow_deepening  
    + tiny_highlight_preservation
```

Example recipe for `slider = +50`:

```
Exposure:   +0.10 EV  
Midtones:   +0.25 tone lift  
Shadows:    +0.15 EV
```

```
Highlights: -0.10 EV compression
Whites:     -0.03 protection
```

Example recipe for `slider = -50`:

```
Exposure:   -0.10 EV
Midtones:   -0.25 tone shift
Blacks:     -0.05 deeper black
Highlights: +0.03 preservation
```

The top-level Brightness slider can secretly call several lower-level operators. That is not cheating; that is good product design.

Recommended UI range

`-100` to `+100`, mapped to approximately:

`-0.45 to +0.45 normalized tone displacement at peak effect`

Guardrails

- Never let Brightness create more highlight clipping than Exposure at the same perceived strength.
- Preserve hue by changing luminance and scaling RGB, not adding white.
- Positive Brightness should not reduce local contrast too much; add a very small contrast compensation if the image becomes flat.
- On already high-key images, reduce highlight movement and concentrate on lower midtones.
- On low-key images, preserve black anchoring so the mood is not destroyed.

8. Gamma slider

User contract

Gamma is an advanced midtone remapping control. It should brighten or darken midtones while preserving black and white endpoints.

Do not expose this in a simple consumer UI unless your users expect technical controls. Most users want Brightness, not Gamma.

Recommended behavior

Operate on normalized tone `x`:


```
gamma = 2 ^ (slider / 100)
x' = x ^ (1 / gamma)
```

This gives:

```
slider +100: gamma = 2.0, midtones brighter
slider 0:    gamma = 1.0, unchanged
slider -100: gamma = 0.5, midtones darker
```

Safer range

Consumer advanced panel:

```
gamma = 0.70 to 1.45
```

Technical panel:

```
gamma = 0.50 to 2.00
```

Guardrails

- Apply to luminance, not each RGB channel independently.
- Keep endpoints fixed.
- Consider blending the effect away from extreme highlights and shadows.
- Do not use Gamma as the implementation for all brightness-like controls. Gamma has a recognizable look and can make images plasticky if overused.

9. Contrast slider

User contract

Contrast should make separation stronger or weaker, mainly in midtones. Increasing contrast should darken middle-dark areas and lighten middle-light areas, while preserving endpoints and avoiding immediate clipping.

Do not implement as

```
rgb = (rgb - 0.5) * contrast + 0.5
```

This clips, shifts color, and treats gamma-encoded values as if they were perceptually sensible.

Recommended behavior

Use an S-curve on normalized tone x .

Simple curve:

$$\begin{aligned} u &= x - 0.5 \\ x' &= x + k * u * (1 - 4u^2) \end{aligned}$$

Where:

$$k = 0.75 * \text{sign}(\text{slider}) * \text{abs}(\text{slider} / 100)^{1.35}$$

This preserves endpoints because the added term approaches zero near $x = 0$ and $x = 1$.

Positive contrast:

- Darkens lower midtones.
- Brightens upper midtones.
- Leaves extreme black/white mostly alone.

Negative contrast:

- Compresses midtones toward the center.
- Should not simply make the whole image gray.

Better version

Use a pivot based on image median instead of fixed 0.5:

$$\begin{aligned} \text{pivot} &= \text{clamp}(\text{percentile_tone_50}, 0.35, 0.65) \\ u &= x - \text{pivot} \end{aligned}$$

This makes contrast behave better on high-key and low-key photos.

Recommended range

Consumer:

$$k = -0.55 \text{ to } +0.75$$

Advanced:

```
k = -0.80 to +1.10
```

Guardrails

- Add highlight and shadow protection after strong positive contrast.
- Keep skin contrast moderate unless the user is clearly asking for grit.
- Do not increase saturation as an accidental side effect. If contrast makes colors appear more saturated, compensate slightly in chroma or provide a separate “vibrance follows contrast” setting.
- Negative contrast should preserve black and white points unless the user is intentionally creating a faded look.

10. Highlights slider

User contract

Highlights should affect bright regions, especially skies, clouds, bright clothes, shiny objects, and light sources. Pulling Highlights left should recover or reveal highlight detail when possible. Pushing it right should brighten highlights without instantly clipping them.

Recommended behavior

Use a luminance-zone mask:

```
w = smoothstep(0.55, 0.88, x_mask)
```

Where `x_mask` comes from a blurred or edge-aware luminance guide, not raw pixel luminance.

Apply exposure-like adjustment in log luminance:

```
EV = max_highlight_EV * sign(slider) * abs(slider / 100)^1.35  
logY' = logY + EV * w * protection
```

Suggested range:

```
negative: down to -1.5 EV  
positive: up to +0.8 EV
```

Protection term:

```
protection = 1 - smoothstep(0.92, 1.00, x)
```

for positive values, so near-white pixels are not pushed hard into clipping.

Guardrails

- Pulling Highlights left cannot recover detail that does not exist. If all channels are clipped, you can only make the clipped region darker, not truly detailed.
- If one or two channels are clipped but another remains usable, advanced reconstruction can borrow detail from unclipped channels.
- Avoid gray, dead highlights. Strong highlight recovery often needs small local contrast restoration.
- Protect skin. Foreheads and cheeks are often highlights, but users usually do not want them flattened.

11. Shadows slider

User contract

Shadows should open or deepen dark regions without moving the whole black point. Positive Shadows should reveal detail. Negative Shadows should add depth and mood.

Recommended behavior

Use a shadow mask:

```
w = 1 - smoothstep(0.18, 0.48, x_mask)
```

Apply exposure-like movement:

```
EV = max_shadow_EV * sign(slider) * abs(slider / 100)^1.35  
logY' = logY + EV * w * black_anchor * noise_protection
```

Suggested range:

```
positive: up to +1.5 EV  
negative: down to -1.0 EV
```

Black anchoring:

```
black_anchor = smoothstep(0.02, 0.15, x)
```

This prevents pure black from becoming gray.

Noise protection:

```
noise_protection = 1 - noise_mask_strength
```

where noisy, very dark regions receive less lift unless denoising is also applied.

Guardrails

- Positive Shadows should not lift black borders, letterboxes, or true empty black.
- Shadow lifting should reduce chroma slightly in very dark areas to avoid colored noise.
- Local contrast should be preserved. Use a guided mask or edge-aware base layer.
- If shadows look flat after lifting, add mild local contrast only to lifted regions.

12. Whites slider

User contract

Whites sets the upper endpoint or white clipping point. It controls how brilliant the brightest tones feel.

Recommended behavior

This is closer to Levels than to Highlights.

Use robust white percentile:

```
white_source = percentile(Y, 99.5 or 99.8)
```

Map the upper range toward a target output. Positive Whites moves more pixels toward output white; negative Whites compresses the top end.

Suggested range:

```
-0.75 EV to +0.75 EV equivalent
```

Positive Whites

Positive Whites may intentionally clip small specular highlights. That is acceptable and often desirable.

Target:

```
allow 0.05% to 0.3% clipping depending on image type
```

Negative Whites

Negative Whites should reduce clipping and tame harsh highlights, but not make the whole photo dull. It should mostly affect the upper tonal endpoint, not all bright midtones.

Guardrails

- Do not let Whites replace Highlights. Highlights is a regional bright-tone control; Whites is endpoint control.
- Avoid per-channel clipping. Preserve hue in bright saturated regions.
- If the user increases Whites, use a shoulder curve rather than hard clipping.

13. Blacks slider

User contract

Blacks sets the lower endpoint or black clipping point. It controls depth, anchoring, and perceived richness.

Recommended behavior

Use robust black percentile:

```
black_source = percentile(Y, 0.2 or 0.5)
```

Negative Blacks maps more low tones to near-black. Positive Blacks lifts the black point or reduces clipping.

Suggested range:

```
-0.75 EV to +0.75 EV equivalent
```

Guardrails

- Negative Blacks should not crush all shadow detail.
- Positive Blacks should not produce a gray veil unless the user is intentionally creating a faded look.
- When lifting blacks, consider reducing saturation in the lifted near-black range to avoid ugly color noise.
- For filmic looks, pair lifted Blacks with a gentle contrast or clarity increase so the image does not become washed out.

14. Tone Curve

User contract

Tone Curve is the advanced manual version of Brightness, Contrast, Highlights, Shadows, Whites, and Blacks.

Recommended modes

Offer two curve modes:

1. **Parametric curve:** Shadows, Darks, Lights, Highlights sliders.
2. **Point curve:** user places points manually.

For the parametric curve, define tonal regions with adjustable boundaries:

```
Shadows:    0.00–0.25
Darks:      0.15–0.50
Lights:     0.50–0.85
Highlights: 0.75–1.00
```

Blend regions smoothly. Moving one region should push neighboring regions gently, not create kinks.

Guardrails

- Enforce monotonicity by default. A non-monotonic curve can create solarization-like effects, which belongs in an advanced or creative mode.
- Smooth the curve unless the user explicitly requests hard points.
- Do not allow accidental endpoint clipping without clear feedback.
- If editing RGB curves independently, label this as color grading, not ordinary tone correction.

15. Saturation slider

User contract

Saturation should make colors more or less colorful globally.

Do not implement in HSV

HSV saturation changes often produce bad hue shifts and unpleasant highlight behavior.

Recommended behavior

Convert to OKLCH or similar:

```
L = perceptual lightness
C = chroma
h = hue
```

Then:

```
C' = C * scale
```

Suggested mapping:

```
u = slider / 100

if u >= 0:
    scale = 1 + 0.65 * u^1.35
else:
    scale = 1 + u
```

This means:

```
+100 saturation => 1.65x chroma
-100 saturation => grayscale
```

Guardrails

- Reduce positive saturation in highlights.
- Reduce positive saturation near gamut boundaries.
- Preserve skin unless the user uses a targeted color tool.
- Apply gamut mapping after chroma increase.
- Avoid clipping channels independently.

A useful protection:

```
sat_boost *= (1 - 0.6 * smoothstep(0.75, 1.0, x))
sat_boost *= (1 - 0.8 * gamut_risk)
```

16. Vibrance slider

User contract

Vibrance should make weak colors richer while avoiding cartoon color, skin damage, and channel clipping.

Recommended behavior

Vibrance is selective saturation.

In OKLCH:


```
C_norm = C / estimated_max_chroma_for_this_lightness_and_hue
low_chroma_weight = (1 - C_norm)^1.5
skin_protection = 1 - 0.7 * skin_mask
highlight_protection = 1 - 0.5 * smoothstep(0.75, 1.0, x)

C' = C * (1 + amount * low_chroma_weight * skin_protection *
highlight_protection)
```

Suggested range:

```
amount = -0.50 to +0.75
```

Positive Vibrance should do very little to already intense reds, oranges, neon signs, and saturated flowers. It should do more to pale skies, foliage, muted clothing, and weak environmental colors.

Negative Vibrance can reduce colorfulness while preserving some strong colors, which often looks better than straight desaturation.

Guardrails

- Skin mask should be soft and conservative.
- Keep hue stable.
- Do not let Vibrance act as hidden contrast.
- Use Saturation for global chroma; use Vibrance for protected chroma.

17. Warmth / Temperature slider

User contract

Warmth should make the image warmer or cooler. Users expect orange/blue correction, not random color tinting.

Recommended behavior

For raw files, adjust white balance before demosaicing or early in linear RGB using camera color information.

For rendered JPEG/HEIC files, use chromatic adaptation or a perceptual approximation.

Use mireds rather than linear Kelvin. A mired is one million divided by color temperature in Kelvin. Equal mired steps feel more visually even than equal Kelvin steps.

Suggested range:

```
consumer: ±40 mired  
advanced: ±100 mired
```

Approximate behavior:

- Positive Warmth: shift neutral axis toward amber.
- Negative Warmth: shift neutral axis toward blue.

Guardrails

- Preserve neutral grays as much as possible.
- Do not over-warm skin while destroying white clothing or walls.
- If the image has mixed lighting, global Warmth will not solve everything. Local white balance or subject-aware correction is needed.
- Avoid pushing saturated blues and oranges out of gamut.

18. Tint slider

User contract

Tint corrects green/magenta color casts.

Recommended range

```
consumer: ±30 perceptual tint units  
advanced: ±60
```

Recommended behavior

Operate in an opponent color space. In OKLab-like terms, shift along the green/magenta axis while preserving luminance.

Guardrails

- Strong Tint should not change exposure.
- Protect skin from becoming radioactive magenta.
- Use neutral-region detection for auto tint, not the whole image average.
- Fluorescent and LED lighting can have discontinuous spectra; a single Tint slider may not fully fix them.

19. Clarity slider

User contract

Clarity adds or removes midtone local contrast. It should make dull images punchier and textured, but it is not the same as sharpening.

Recommended behavior

Use edge-aware local contrast on luminance.

Basic decomposition:

```
base = edge_aware_blur(logY, radius = 20-80 px)
detail = logY - base
```

But Clarity should not boost all detail equally. It should focus on broad local contrast, not pixel-level sharpness.

A better decomposition:

```
base_large = edge_aware_blur(logY, radius = 60 px)
base_small = edge_aware_blur(logY, radius = 8 px)
midtone_detail = base_small - base_large
```

Then:

```
logY' = logY + amount * midtone_detail * midtone_weight * protection
```

Suggested range:

```
amount = -0.35 to +0.45
```

where is strong but not halo city.

Midtone weighting

```
midtone_weight = exp(-((x_mask - 0.5)^2) / (2 * 0.25^2))
```

This keeps Clarity mostly in midtones.

Protection

```
protection =  
    (1 - 0.7 * skin_mask)  
    * (1 - 0.7 * sky_smooth_mask)  
    * (1 - 0.5 * noise_mask)
```

Negative Clarity

Negative Clarity should soften local contrast, especially in midtones. It can be useful for portraits. But do not simply blur the image. Reduce midtone local contrast while preserving edges and fine detail where possible.

Guardrails

- Avoid halos along skylines, buildings, tree branches, and high-contrast edges.
- Avoid making skin pores and wrinkles harsh.
- Avoid increasing noise in shadows.
- Avoid global darkening or saturation shifts.
- At +100, a cityscape may look punchy; a face should not look diseased.

20. Texture slider

User contract

Texture affects fine-to-medium surface detail. It should make hair, fabric, bark, paper, grain, and small repeated structures stand out or soften. It should not dramatically change global contrast.

Difference from Clarity

- Texture: smaller radius, fine detail, less tonal drama.
- Clarity: larger radius, midtone local contrast, more punch.
- Sharpening: edge acuity and deconvolution-like crispness.

Recommended behavior

Use a smaller edge-aware detail layer:

```
base = edge_aware_blur(logY, radius = 2-8 px)  
fine_detail = logY - base  
logY' = logY + amount * fine_detail * protection
```

Suggested range:

```
amount = -0.20 to +0.30
```

Guardrails

- Do not boost sensor noise.
- Do not over-texture faces by default.
- Do not create crunchy edges.
- Positive Texture should have almost no effect on flat skies and smooth gradients.
- Negative Texture should soften surfaces without making the image look out of focus.

21. Dehaze slider

User contract

Dehaze should cut through atmospheric haze, fog, smoke, or flat bright veiling. Negative Dehaze should add haze.

This is a dangerous slider. It can easily create black skies, oversaturated color, blocked shadows, and crunchy halos.

Recommended behavior

A simplified dehaze model:

```
observed = scene * transmission + airlight * (1 - transmission)
```

Estimate:

- `airlight`: color and brightness of haze veil.
- `transmission`: how much scene detail survives through the haze.

Then recover:

```
scene_est = (observed - airlight) / max(transmission, t_min) + airlight
```

For a consumer implementation, do not try to perfectly solve physical haze. Use a haze-likelihood mask and apply a coordinated recipe:

For positive Dehaze:

```
Highlights:  -0.15 to -0.40 EV in hazy bright regions
Contrast:    +small S-curve
Clarity:     +small local contrast
```

```
Saturation:  +small protected chroma boost
Black point:  slight deepening
```

For negative Dehaze:

```
Contrast:     lower
Black point:  lift
Highlights:   soften
Saturation:   reduce slightly
Warmth:       optionally shift warmer
```

Suggested range:

```
consumer: -50 to +50
advanced: -100 to +100
```

Even if the UI shows ± 100 , internally cap the effect based on haze confidence.

Guardrails

- If haze confidence is low, attenuate the slider.
- Protect faces.
- Protect blue skies from turning navy or cyan sludge.
- Protect saturated colors from clipping.
- Avoid halos around mountains, trees, buildings, and horizon lines.
- Do not let Dehaze become “instant HDR ugliness.”

22. Pop slider

User contract

Pop should make the image more lively in one move. It is not a fundamental image operation. It is a macro.

Recommended recipe

For positive Pop:

```
Contrast:  +0.10 to +0.20
Clarity:   +0.08 to +0.18
Vibrance:  +0.08 to +0.20
Whites:    +0.03
```

Blacks: -0.03
Highlights: -0.03 protection

For negative Pop:

Contrast: -0.10
Clarity: -0.08
Vibrance: -0.05
Blacks: +0.03

Guardrails

- Pop should be content-aware.
- On portraits, reduce Clarity and Vibrance.
- On landscapes/cityscapes, allow more Clarity and Dehaze-like behavior.
- On high-contrast images, avoid adding much contrast.
- On already saturated images, avoid adding much color.

23. HDR / Brilliance slider

User contract

This is the “make the image feel more computationally alive” slider. Apple Photos has a control called Brilliance; Google Photos has HDR-like effects. The practical idea is to compress dynamic range while increasing local readability.

Recommended recipe

Positive Brilliance:

Shadows: +0.20 to +0.50 EV
Highlights: -0.10 to -0.40 EV
Brightness: +small midtone lift
Contrast: +small midtone contrast
Clarity: +small local contrast
Vibrance: +small protected boost

Negative Brilliance:

Shadows: -slight
Highlights: +slight

Clarity: reduce
Contrast: reduce

Guardrails

- This slider should not create an HDR grunge look by default.
- Preserve global mood. A night photo should still look like night.
- Preserve skin.
- Avoid flattening highlights into gray.
- Avoid lifting noisy shadows too much.

24. Sharpening slider

User contract

Sharpening should make edges crisper. It should not act like Clarity or Texture.

Recommended behavior

Use an unsharp mask, deconvolution-like method, or edge-aware sharpening.

Basic unsharp mask:

```
blur = gaussian(Y, radius)
detail = Y - blur
Y' = Y + amount * detail
```

Better:

- Sharpen luminance, not RGB channels.
- Use edge masks.
- Avoid sharpening noise.
- Avoid sharpening halos.

Suggested controls:

```
Amount:      0-150%
Radius:      0.5-2.0 px
Detail:      controls high-frequency inclusion
Masking:     protects flat areas
```

For a simple consumer slider, internally map:


```
slider +100:  
  amount  $\approx$  80–120%  
  radius  $\approx$  0.8–1.2 px  
  masking based on noise/flatness
```

Guardrails

- Never sharpen smooth skies.
- Reduce sharpening in high-ISO shadows.
- Avoid color halos by sharpening luminance only.
- Downsample-aware sharpening: preview at different zoom levels should not lie.

25. Vignette slider

User contract

Vignette darkens or lightens edges, usually to focus attention.

Recommended behavior

Use an elliptical radial mask:

```
r = normalized distance from chosen center  
w = smoothstep(inner_radius, outer_radius, r)  
Y' = Y * 2^(EV * w)
```

Suggested range:

EV = -1.5 to +1.0

Controls:

Amount
Midpoint
Feather
Roundness
Center
Highlight preservation

Guardrails

- Do not crush corners to black unless the user asks.

- Preserve bright highlights in corners if highlight preservation is enabled.
- Apply after crop.
- Use perceptual falloff, not a crude radial multiply.

26. Color Mix / HSL sliders

User contract

Targeted color sliders should adjust a color family without damaging neighboring colors.

Recommended behavior

Use hue masks in a perceptual color space.

For each target color:

```
hue_weight = circular_gaussian_distance(pixel_hue, target_hue)
```

Then adjust:

- Hue: rotate hue within mask.
- Saturation: scale chroma within mask.
- Luminance: adjust perceptual lightness within mask.

Guardrails

- Red/orange/yellow overlap affects skin. Be conservative.
- Hue rotations should fall off smoothly.
- Avoid discontinuities near hue wraparound.
- Use gamut mapping after changes.
- Let users see affected regions if possible.

27. Skin Tone slider

User contract

Skin Tone should improve skin without acting like a generic orange slider.

Recommended behavior

Build a soft skin mask using:

```
hue range  
chroma range  
luminance range
```

```
face/person segmentation if available  
texture cues
```

Then allow small shifts:

```
hue:      toward/away from red-yellow balance  
chroma:   modest saturation normalization  
lightness: tiny correction only  
texture:  optional protection from clarity
```

Guardrails

- Never assume all skin has the same hue or lightness.
- Do not lighten or darken skin aggressively without user intent.
- Do not use ethnicity-specific assumptions.
- Keep the mask soft and editable.
- When in doubt, protect skin from other sliders rather than “fixing” it.

28. Auto button

User contract

Auto should make the photo broadly better, not perfect. It should be reversible and decomposed into normal sliders so the user can continue editing.

Recommended behavior

Auto should estimate:

```
exposure correction  
white and black points  
highlight recovery  
shadow lift  
midtone brightness  
contrast  
white balance  
vibrance  
small clarity/pop
```

Then write those as ordinary slider values.

Example Auto recipe:

1. Place midtone around target brightness.
2. Protect highlights.
3. Set white point with small allowed specular clipping.
4. Set black point with small allowed shadow clipping.
5. Lift shadows if dynamic range is high.
6. Add mild contrast if image is flat.
7. Add vibrance if chroma is low.
8. Avoid clarity on portraits.

Guardrails

- Auto should not maximize histogram spread blindly.
- Auto should not make night photos look like daytime.
- Auto should not neutralize intentionally warm sunset light.
- Auto should not crush foggy or high-key images.
- Auto should expose its decisions as editable sliders.

29. Output soft clipping and gamut mapping

Even with good sliders, some values will exceed display range. Handle this gracefully.

Luminance shoulder

For values above a shoulder start:

```
if x <= shoulder_start:
    y = x
else:
    y = shoulder_start + (1 - shoulder_start) * (1 - exp(-(x - shoulder_start) /
softness))
```

Suggested:

```
shoulder_start = 0.85
softness = 0.20
```

Tune by display transform. The idea is to roll highlights toward white rather than slam them into white.

Toe for shadows

For deep shadows, use a gentle toe so near-black values do not all collapse:

```
y = toe_curve(x)
```

A toe curve should preserve black while giving separation just above black.

Gamut mapping

When color changes push RGB outside the target gamut:

Bad:

```
clip each channel independently
```

Better:

```
preserve hue and lightness  
reduce chroma until the color fits
```

This is critical for Saturation, Vibrance, Warmth, Tint, Dehaze, and contrast-heavy edits.

30. HDR and gain maps

Modern phone images may contain a standard image plus HDR luminance metadata, often called a gain map. If your editor ignores this, advanced edits can produce mismatched SDR/HDR results.

Minimum policy:

- For crop/rotate/scale, transform the gain map the same way.
- For global tone edits, apply a corresponding transform to the gain map or regenerate it.
- For local edits, either update the affected gain-map region, neutralize that region, or drop HDR metadata.
- For filters and ML edits, do not blindly preserve the old gain map if the visible image content changed.

Practical first version:

```
If the edit only changes geometry:  
    transform gain map identically.
```

```
If the edit changes global tone/color:  
    update gain map with approximate luminance transform.
```

```
If the edit changes local content:  
    neutralize or recompute gain map in edited regions.
```

If unsure:
export SDR and clearly mark HDR as not preserved.

Do not silently produce ghosted or inconsistent HDR output.

31. Recommended top-level consumer controls

A clean consumer editor should not expose every primitive at once. Use friendly top-level controls backed by expandable advanced controls.

Recommended simple set:

- Auto
- Brightness
- Contrast
- Highlights
- Shadows
- Warmth
- Tint
- Saturation
- Vibrance
- Pop
- Clarity
- Texture
- Sharpen
- Vignette

Optional advanced:

- Exposure
- Whites
- Blacks
- Gamma
- Tone Curve
- Color Mix
- Dehaze
- Noise Reduction
- Grain
- HDR/Gain Map behavior

The important product decision: **Brightness, Pop, and Auto should be macros over several lower-level operators.** Exposure, Whites, Blacks, and Gamma should be closer to technical primitives.

32. Suggested default slider bounds

Use these as starting points.

Exposure:	± 2.0 EV consumer, ± 5.0 EV advanced
Brightness:	peak tone displacement ± 0.45
Gamma:	0.70–1.45 consumer advanced, 0.50–2.00 technical
Contrast:	k -0.55 to +0.75 consumer
Highlights:	-1.5 EV to +0.8 EV
Shadows:	-1.0 EV to +1.5 EV
Whites:	± 0.75 EV endpoint equivalent
Blacks:	± 0.75 EV endpoint equivalent
Saturation:	-100% to +65% chroma scale
Vibrance:	-50% to +75% protected chroma boost
Warmth:	± 40 mired consumer, ± 100 advanced
Tint:	± 30 consumer, ± 60 advanced
Clarity:	-0.35 to +0.45 local contrast amount
Texture:	-0.20 to +0.30 fine detail amount
Dehaze:	± 50 consumer, ± 100 advanced but content-attenuated
Vignette:	-1.5 EV to +1.0 EV edge falloff
Sharpening:	0–120% effective amount, content-masked

These are not sacred values. They are safe starting points. Tune with test images.

33. Acceptance tests for every slider

Create a test suite of images and run every slider at:

-100, -50, -10, +10, +50, +100

Use at least:

1. Backlit portrait.
2. Pale skin portrait.
3. Dark skin portrait.
4. Snow or wedding dress.
5. Blue sky with clouds.
6. Night city with neon signs.
7. Flat foggy landscape.
8. High-contrast midday street.
9. Saturated flowers.
10. Indoor mixed lighting.
11. High-ISO dark image.
12. Smooth gradient.

13. Fine texture: hair, fabric, grass, bark.
14. Film scan or scanned negative conversion output.
15. Screenshot or graphic with hard edges.

For each slider, check:

```
Does +10 visibly do something?  
Does +50 look useful?  
Does +100 remain plausible on some images?  
Does clipping stay controlled?  
Are hues stable?  
Are skin tones protected?  
Are skies protected?  
Are shadows noisy?  
Are halos visible?  
Does before/after match the slider's name?
```

Also compute automated metrics:

```
clipped pixel ratio  
per-channel clipping ratio  
gamut excursions  
mean hue shift in skin mask  
mean chroma change in skin mask  
shadow noise amplification  
halo score near high-contrast edges  
histogram endpoint movement  
local contrast change by frequency band
```

Do not rely only on metrics. Photo editing is perceptual. But metrics catch regressions.

34. Common bugs and their likely causes

“The slider barely does anything”

Likely causes:

- Underlying range too small.
- Slider maps linearly but the operation is perceptually weak.
- Working in linear space for a perceptual adjustment without enough magnitude.
- Preview transform hides the effect.
- Operator only affects a narrow mask.

Fix:

- Use nonlinear slider mapping.
- Increase midtone effect.
- Show numeric diagnostics for peak tone displacement.

“The slider clips all the time”

Likely causes:

- Clamping after each operation.
- Applying RGB addition or multiplication in display space.
- No highlight shoulder.
- No percentile-based clipping target.
- Per-channel clipping instead of luminance-preserving gamut mapping.

Fix:

- Keep float values unclamped.
- Tone-map at output.
- Add shoulder/toe.
- Protect highlights and saturated colors.

“Brightness looks washed out”

Likely causes:

- Lifting black point too much.
- No contrast compensation.
- Adding white instead of moving luminance.
- Flattening local contrast.

Fix:

- Anchor blacks.
- Use midtone curve.
- Add tiny contrast or local contrast compensation.
- Preserve chroma carefully.

“Contrast looks harsh”

Likely causes:

- Linear contrast around 0.5 in RGB.
- No endpoint protection.
- Too much saturation side effect.
- Wrong pivot for high-key or low-key image.

Fix:

- Use S-curve on luminance.
- Use image-adaptive pivot.
- Add highlight/shadow protection.
- Reduce chroma if contrast increases apparent saturation too much.

“Clarity creates halos”

Likely causes:

- Gaussian blur instead of edge-aware blur.
- Radius too large.
- Amount too high.
- No edge protection.
- Applying to raw pixel luminance instead of guided base/detail layers.

Fix:

- Use bilateral, guided, or local Laplacian filtering.
- Limit amount.
- Use halo detection near strong edges.
- Reduce effect in sky/edge transitions.

“Saturation makes colors radioactive”

Likely causes:

- HSV/HSL saturation.
- No gamut mapping.
- Boosting already saturated colors.
- Boosting skin equally.

Fix:

- Use OKLCH-like chroma.
- Make Vibrance selective.
- Protect skin and highlights.
- Reduce chroma to fit gamut instead of clipping channels.

35. A sane first implementation plan

Build in this order:

Phase 1: safe global tone

Implement:

```
float working buffer
linear RGB conversion
luminance computation
Exposure
Brightness
Contrast
Highlights
Shadows
Whites
Blacks
soft output clipping
clipping diagnostics
```

Do not start with filters or AI. Get these right first.

Phase 2: protected color

Implement:

```
OKLab/OKLCH conversion
Saturation
Vibrance
Warmth
Tint
basic skin protection
gamut mapping
```

Phase 3: local detail

Implement:

```
edge-aware blur
Clarity
Texture
Sharpening
noise-aware masks
halo tests
```

Phase 4: macros

Implement:

```
Auto
Pop
Brilliance/HDR effect
portrait-aware defaults
landscape-aware defaults
```

Macros should write ordinary slider values so users can inspect and adjust the result.

Phase 5: modern formats

Implement:

```
HDR/gain-map preservation policy
wide-gamut export
HEIC/Ultra HDR behavior
metadata handling
edit stack serialization
copy/paste edits
```

36. The design target

The editor is good when the simple sliders feel almost boringly reliable.

A user should be able to drag Brightness to +40 and think, “Yes, that is what I meant.” They should be able to drag Contrast to +30 and get more separation without the sky exploding. They should be able to drag Shadows to +50 and see detail without gray sludge. They should be able to add Vibrance without destroying skin. They should be able to use Clarity without halos.

That reliability comes from treating each slider as an intelligent, guarded photo operation rather than a direct mathematical knob.

The UI can be simple only if the backend is not.